ATELIER 5 Où est mon colis

Algorithmie comparative - J3 - Mardi 10 novembre 2026, 10h10 - 35 minutes, en binôme

La question à laquelle l'atelier répond

La page Où est mon colis reçoit 40 000 requêtes par jour, dont 60 % entre 8h et 12h. Quelle structure lui faut-il pour retrouver le dernier scan d'un colis, et que coûte-t-elle ?

Ce que vous produisez : adr/GRILLE-ADR-002.md et adr/ADR-002-suivi.md (carnet), deux CSV et une courbe dans bench/resultats.

Le rendu attendu

En fin d'atelier : le code, les mesures et la grille passent le correcteur, et l'ADR existe en premier jet (chaque rubrique en une à trois lignes, avec vos chiffres et le nom de vos CSV). Le correcteur sans KO avant 17h00, le commit du binôme à 17h00. Ensuite, l'ADR s'améliore chez vous jusqu'au dépôt du carnet, le 1er décembre.

Avant 17h00, le correcteur de l'atelier répond sans KO :

```
OK  À COMPLÉTER : la Map rend le même dernier scan que le legacy
OK  Le coût d'une requête est mesuré jusqu'au jeu grande
OK  Le coût de construction est mesuré, mémoire comprise
OK  La courbe des requêtes est tracée
OK  La grille comparative de l'ADR-002 est remplie
OK  L'ADR-002 a ses six rubriques, remplies
OK  La rubrique 3 de l'ADR-002 renvoie à une mesure qui existe, la rubrique 5 donne le domaine de validité
OK  Les trois phrases de l'ADR-002 tiennent sans jargon
8 contrôles, 0 KO
```

Avant de commencer. Le code des ateliers 5 à 7 est dans votre dépôt (git pull cours j3 hier soir, ou maj-J3.zip décompressé à la racine) et le jeu grande d'hier est présent (data/jeux/grande). Copiez la grille. Écrivez-y tout de suite les poids et les cinq barèmes du sous-objectif 5 :

Windows	`Copy-Item adr\grille-comparative.md adr\GRILLE-ADR-002.md`
macOS, Linux	`cp adr/grille-comparative.md adr/GRILLE-ADR-002.md`

Aujourd'hui, la fiche atelier en cours donne la première commande de chaque sous-objectif et ce que vous devez voir. Les autres commandes sont celles des fiches atelier numéro 2 à 4, et toutes sont dans le README du dépôt. Les consignes de la slide sont rappelées en face de chaque sous-objectif.

Les sous-objectifs

1. Lire le legacy de la page

consigne 1 de la slide

Pourquoi : Le prix d'une requête vient d'une ligne qui grandit avec les scans : 996 844 dans le jeu grande, 13 millions en base.

Faites : Retrouvez dans legacy/src/suivi.ts la ligne que l'animation a montrée, puis lisez la fonction qui la contient : votre binôme doit pouvoir dire pourquoi elle grandit avec le nombre de scans.

Windows	`Select-String -Path legacy\src\suivi.ts -Pattern "filter"`
macOS, Linux	`grep -n "filter" legacy/src/suivi.ts`

Vous devez voir : La ligne de l'animation, avec son numéro. La fonction qui la contient dit ce qu'elle fait à chaque requête.

2. Compléter la Map, lancer les tests

consigne 2 de la slide

Pourquoi : La Map doit rendre le même dernier scan que le legacy ; la fonction plusRecent existe déjà.

Faites : Complétez l'unique À COMPLÉTER de construireMap, dans bench/variantes/suivi.ts, puis lancez les tests.

Les trois systèmes	`npm test`

Vous devez voir : Les deux tests de suivi passent. Ceux du tas et du rapport échouent encore : ils attendent les ateliers 6 et 7.

```
✔ les variantes de suivi rendent le même dernier scan que le legacy (115.2895ms)
✔ un numéro inconnu rend null, une liste vide aussi (0.1466ms)
```

3. Mesurer le coût d'une requête consigne 3 de la slide

Pourquoi : Cent requêtes par appel, dont une sur cinq porte un numéro mal tapé, de 10 000 au million de scans.

Faites : Lancez la mesure (une à deux minutes selon le poste), puis tracez la courbe avant la mesure de construction : npm run courbe trace le CSV le plus récent.

Les trois systèmes	`npm run bench -- --sujet suivi --repetitions 5 --echauffement 3`
puis	`npm run courbe -- --log`

Vous devez voir : Au million de scans, la Map répond des milliers de fois plus vite que le tableau. Vos durées seront différentes, l'écart non. Divisez par 100 pour le prix d'une requête.

```
variante        taille   duree_ms   memoire_ko
tableau         996844    2686.17           63
set             996844    1776.67           59
map             996844       0.04            2
```

4. Mesurer le coût de construction consigne 4 de la slide

Pourquoi : La Map se paie une fois, en temps et en mémoire ; l'ADR dira au bout de combien de requêtes elle est remboursée.

Faites : Lancez la mesure de construction (une à deux minutes).

Les trois systèmes	`npm run bench -- --sujet suivi-construction --repetitions 5 --echauffement 3`

Vous devez voir : La colonne memoire_ko : quelques Mo pour la Map et le Set au million de scans, presque rien pour le tableau.

```
variante        taille   duree_ms   memoire_ko
tableau         996844       0.02            1
set             996844     222.93         5122
map             996844     685.31         7170
```

5. Remplir la grille, écrire l'ADR-002 consignes 5 et 6 de la slide

Pourquoi : Les poids de la page : temps 5, mémoire 2, lisibilité 1, robustesse 1, coût 1. Temps, par requête projetée sur les six mois de la base : 5 sous 1 ms, 4 sous 10 ms, 3 sous 100 ms, 2 sous 1 s, 1 au-delà. Mémoire, la structure construite au million de scans : 5 sous 1 Mo, 4 sous 10 Mo, 3 sous 50, 2 sous 200, 1 au-delà ; memoire_ko est un ordre de grandeur, et deux relevés à moins d'un facteur 2 l'un de l'autre prennent la même note. Lisibilité : vous lisez un code que vous n'avez pas écrit ; partez de 5 et retirez un point par idée qu'il faut connaître pour le lire (la table de hachage sous le Set et la Map). Robustesse : 5 si tout numéro, connu, inconnu ou mal formé, reçoit la bonne réponse ou une erreur claire ; 4 si la structure n'a rien à tenir à jour mais ne vérifie pas le format du numéro ; 3 si la réponse dépend d'une structure à tenir à jour, qu'un scan manqué rend périmée sans erreur ; 2 si un cas courant donne un résultat faux en silence ; 1 si npm test échoue. Coût : 5 si rien ne change ; 4 pour quelques lignes et une structure fournie par le langage ; 3 pour une structure à écrire et à tester ; 2 pour une donnée à préparer à chaque lancement ; 1 pour une dépendance ajoutée.

Faites : Remplissez la grille. Pour le temps, divisez la médiane à 996 844 scans par 100, puis multipliez-la par 13 pour le tableau et le Set, qui relisent tous les scans ; la Map ne grandit pas avec les scans. Copiez le gabarit d'ADR, puis écrivez l'ADR-002. Préparez la décision et la question de compréhension à deux, puis lancez `npm run verifier -- 5`.

Windows	`Copy-Item adr\ADR-000-gabarit.md adr\ADR-002-suivi.md`
macOS, Linux	`cp adr/ADR-000-gabarit.md adr/ADR-002-suivi.md`

- En atelier, l'ADR est un premier jet ; ce qui manque se reprend cet après-midi, dans le créneau de l'atelier 7, puis vous la finissez chez vous, pour le carnet du 1er décembre.
- Rubrique 3 : deux tableaux, les requêtes (10 000, 100 000 et 996 844 scans) et la construction (temps et mémoire à 996 844), à la place des colonnes du gabarit ; vos deux CSV.
- Rubrique 5 : la mémoire de la Map sur six mois (elle garde une case par colis, pas par scan) et le jour où elle n'est pas à jour.
- La sensibilité va en section 3 de la grille : vos poids sont déjà ceux qu'elle propose ; écrivez à partir de quel poids du temps la Map perd. Si la décision est fragile, une phrase en rubrique 5. npm run couples ne sert pas aujourd'hui.

Vous devez voir : Avant 17h00, le correcteur sans KO, comme en première page : 8 contrôles, 0 KO. Il ne dit pas si la décision est bonne.

La décision de l'atelier

Quelle structure retenez-vous pour la page, et quel critère a tranché, avec un chiffre mesuré ? C'est la rubrique 4 de l'ADR-002, en une phrase.

La question de compréhension

À quoi sert ce que vous venez de faire, en deux phrases ? Préparez-la à deux : en fin d'atelier, l'intervenant passe la demander à quelques binômes.

Au besoin

Où trouver un exemple dans le code

bench/variantes/suivi.ts contient les trois variantes de l'atelier (tableau, set, map) et une quatrième, dichotomie, qui sert cet après-midi, au cours 6 ; tableau et set montrent comment une variante construit puis cherche. legacy/src/suivi.ts contient plusRecent. bench/sujets/suivi.ts montre comment les requêtes sont tirées.

Si ça coince

- Le test "les variantes de suivi rendent le même dernier scan que le legacy" échoue : la Map garde le premier scan rencontré ; utilisez plusRecent.
- Le harness s'arrête sur ARRÊT à 10 000 scans : la Map n'est pas complétée, ou rend autre chose que le legacy. L'ARRÊT n'arrive qu'après la mesure complète du tableau et du Set (une bonne minute) : attendez-le, ou relancez avec --tailles 10000.
- "jeu introuvable" : le jeu grande n'est pas généré ; lancez `npm run data -- --echelle grande` (une demi-minute).

Toujours bloqué après ces trois pistes : levez la main, ou écrivez sur Teams.

Si vous avez fini

- Mesurez avec 50 % de numéros mal tapés (la proportion se règle dans genererRequetes, bench/variantes/suivi.ts), puis comparez le Set au tableau. Remettez 0,8 ensuite.
- Comptez combien des 40 000 requêtes d'une journée le Set écarterait sans rien relire, puis écrivez en rubrique 5 si cela justifie de le garder à côté de la Map.

Annexe : les cas particuliers

La courbe des requêtes se trace juste après leur mesure, avant celle de la construction : npm run courbe trace le CSV le plus récent. Pour en tracer un autre, donnez son chemin après --log (README du dépôt).

```
Les trois systèmes      npm run courbe -- --log
```

```
courbe écrite : bench\resultats\2026-11-08T17-28-39-suivi.svg
```

Le test de suivi est ignoré (skip) : le jeu petite n'existe pas. Générez-le.

```
Les trois systèmes      npm run data -- --echelle petite
```

Les sorties montrées sont de vraies sorties, capturées en jouant la fiche atelier en cours ; vos durées seront différentes, vos nombres de lignes, de clients et vos sommes de contrôle non. Avant 17h00, le binôme commite : git add -A puis git commit -m "J3". Sans commit, le travail du jour n'est pas évalué.